

703088 (2026WS) - PS Distributed Systems

Distributed and Parallel Systems Group
University of Innsbruck
Instructors: Juan Aznar Poveda, Zahra Najafabadi Samani, and Marlon Etheredge

EX01 - Cloud Infrastructure

Introduction

Modern distributed applications rely heavily on diverse cloud execution paradigms, ranging from dedicated Virtual Machines (VMs) and containerized environments to serverless functions. Understanding how these foundational cloud primitives operate is the first step toward building scalable systems.

In this first exercise, you will bridge basic cloud infrastructure and infrastructure-as-code (IaC) principles by working with Amazon Web Services (AWS) virtual machines (Amazon EC2) using Terraform, alongside containerization via Docker. You will also have the opportunity to optionally explore serverless offloading using AWS Lambda.

The objective is to familiarize yourself with provisioning infrastructure programmatically, deploying applications across different cloud abstraction layers, measuring execution metrics, and managing containerized resources effectively without getting overwhelmed by heavy orchestration overhead.

Prerequisites

You should have:

- Python 3 installed;
- Docker installed and running;
- Terraform installed for provisioning infrastructure;
- basic knowledge of Python and REST APIs;
- access to AWS Academy (invitation to be sent by the instructor).

Application

For this exercise, you are encouraged to bring your own application or choose a sample project to containerize and deploy onto your cloud infrastructure. The application should ideally be a simple web service, API, or multi-component tool (written in Python, Node.js, Go, or any preferred stack) that you can package into a Docker container and run remotely. If you need inspiration, a starting template, or guidance on how to structure your deployment, you can explore the following curated repositories filled with container-ready sample applications and documentation:

- Docker Awesome Compose: awesome-compose - A massive collection of multi-container sample applications combining web frontends, APIs, and databases.
- Docker Init Demos: docker-init-demos - A clean collection of un-containerized baseline web apps across multiple programming languages, perfect for practicing writing Dockerfiles from scratch.
- Docker Example Voting App: example-voting-app - Docker's flagship multi-service microservices application for advanced multi-container deployments.
- Docker Zero to Hero: Docker-Zero-to-Hero - A comprehensive DevOps training repository containing practical, real-world application codebases ready for containerization.

Task 1 - Virtual Machines and Baseline Execution

Amazon Elastic Compute Cloud (EC2) allows users to rent virtual computers on which to run their own applications. Instead of managing instances manually through the AWS Console, you will use Terraform to provision your infrastructure as code.

1. Provision an EC2 Instance with Terraform: Write a Terraform configuration (`main.tf`) to spin up an EC2 instance with the following specifications:

Parameter	Value
Region	`us-east-1`
AMI	`Amazon Linux 2023 AMI`
Instance Type	`t2.micro`

   - *Note:* Make sure to also define a security group using Terraform that allows the appropriate access rights (e.g., inbound SSH access).
   - Run `terraform init`, `terraform plan`, and `terraform apply` to create your infrastructure, and make note of the generated public IP output.
2. Setup & Execution: Connect to your provisioned VM via SSH, copy your application files over using SCP, and run your baseline application.
3. Measure Performance: Measure the duration of your application using `time` across different workload or depth parameters. Document your Terraform code, commands, and results.

   IMPORTANT: Remember to properly destroy your infrastructure using 'terraform destroy' after completing the task to avoid unexpected charges!

Task 2 - Containerization with Docker

Deploying raw scripts directly on virtual machines can lead to environment drift and dependency conflicts. Containerization simplifies application packaging, distribution, and execution consistency.

1. Create a Dockerfile: Package your application code and its dependencies into a custom Docker image.
2. Build and Run on EC2: Transfer or build your Docker image directly on an EC2 instance, and run the application inside a container. Verify its correctness by checking the container logs.
3. Parallel Containers: Start two container instances in parallel on your EC2 instance. Verify that both containers are running concurrently and check their isolation and process management.

Task 3 - Serverless Offloading with AWS Lambda

AWS Lambda allows you to run code without provisioning or managing servers, scaling automatically with demand.

1. Create a Lambda Function: Create a serverless function that handles a specific computational or data-processing task from your application. Ensure its input and output payloads use JSON.
2. Connect VM/Container to Lambda: Modify your application (running on your EC2 instance or inside your Docker container) to offload part of its workload to your newly created AWS Lambda function via its API.
3. Measure & Compare: Measure the execution time of this hybrid approach using time and compare it against your baseline VM execution from Task 1.

Questions

1. What are the primary benefits of containerizing your application with Docker compared to deploying directly onto a raw EC2 virtual machine?
2. Why is running multiple containers in parallel on a single VM more efficient or manageable than running multiple raw Python processes manually?
3. What are the operational trade-offs between managing code execution on an EC2 VM versus offloading it to a serverless function like AWS Lambda? What did you observe regarding startup times or execution overhead?

Learning objectives

After completing this exercise, you should be able to:

- provision and configure basic AWS cloud resources (EC2);
- containerize and run multi-container applications using Docker;
- measure and analyze performance differences across cloud execution paradigms.

Submission

Submit a PDF file with your solution and your code. You can optionally use the markdown template provided in OLAT (`Submission_Template.md`) and convert it to PDF.

Your report should address all tasks and questions specified in the exercise. Depending on the exercise, this may include:

- theoretical questions and explanations;
- a description of your implementation;
- source code snippet;
- experimental results, observations, or screenshots;
- answers to the discussion questions.

Focus your report on the **distributed-systems concepts, your design decisions, and your observations**. Keep the report concise and avoid documenting routine installation steps unless they are relevant to the exercise.

The submission should follow the following structure and naming convention:

```
|-- exc0X_report_csXXXXXX.pdf
`-- exc0X_source_csXXXXXX.zip
    |-- README.md
    |-- sample_file_a.py
    |-- sample_file_b.py
    `-- sample_file_c.py
```

Deadline

22.10.2026 12:00h

Presentation

23.10.2026